

Ik heb de eerste analyse van de **ClimateChecker-server** uitgevoerd. Hier zijn de belangrijkste bevindingen en inzichten:

1. Architectuur Overzicht

Technologie Stack

- **Taal & Framework:** De codebase is geschreven in **Java** met **Spring Boot** (geen Node.js/Express, Fastify, NestJS, of Koa).
- **Authenticatie:** Gebruikt **JWT-achtige tokens** (maar niet de `jwt`-library; zelf geïmplementeerd via `TokenService`).
- **Database:** **MariaDB** (via Flyway migrations).
- **Build Tool:** **Maven** (gebaseerd op de directory-structuur en `.java`-bestanden).
- **Testing:** **JUnit** (unit tests aanwezig).

Directory Structuur (Hoofdcomponenten)

```
ClimateChecker/
├── src/
│   ├── main/
│   │   ├── java/Ontdekstation013/ClimateChecker/
│   │   │   ├── config/           # Configuratie (Security, CORS, etc.)
│   │   │   ├── features/        # Modulaire opzet per feature (bijv.
│   │   │   │   ├── user/, `e2e`)
│   │   │   │   └── user/
│   │   │   │       ├── authentication/ # Authenticatie-logica (services,
│   │   │   │       │   endpoints, tokens)
│   │   │   │       ├── model/         # Data-modellen (User, Token, etc.)
│   │   │   │       └── endpoint/      # API-endpoints (Controllers)
│   │   │       ├── Application.java   # Main entrypoint (Spring Boot)
│   │   │       └── resources/
│   │   │           ├── db/migration/  # Flyway database migrations
│   │   │           └── application.properties # Configuratie (bijv. database, server
│   │   │               poort)
│   └── test/                       # Unit tests
```

API Opzet

- **RESTful Endpoints:** Gedefinieerd in controllers (bijv. `UserAuthenticationController`).
- **HTTP-methoden:** Gebruikt `GET`, `POST`, `PUT` (standaard REST-patronen).
- **Authenticatie:** Endpoints zoals `/api/my-account/**` vereisen authenticatie (via `SecurityConfiguration`).
- **CORS:** Ingeschakeld via `cors()` in `SecurityConfiguration`.

2. REST Best Practices Evaluatie

✓ Goed Toegepast

- **Resource Naming:** Consistente naming (bijv. `/api/user`, `/api/my-account`).
- **HTTP-methoden:** Correct gebruik van `GET`, `POST`, `PUT`.
- **Statelessness:** JWT-achtige tokens worden gebruikt (stateless authenticatie).
- **Error Handling:** HTTP-statuscodes worden gebruikt (bijv. `401 Unauthorized`, `200 OK`).

⚠ Verbeterpunten

Probleem	Impact	Oplossing
Geen API-versiebeheer	Breaking changes zijn moeilijk	Voeg versiebeheer toe (bijv. <code>/api/v1/user</code>).
Geen OpenAPI/Swagger	Slechte documentatie	Implementeer SpringDoc OpenAPI of Swagger UI voor interactieve API-documentatie.
Geen Pagination	Performance-problemen bij grote datasets	Voeg pagination toe (bijv. <code>?page=1&size=10</code>).
Inconsistente Responses	Verwarring voor clients	Gebruik een standaard response-format (bijv. <code>{ data: ..., meta: ..., errors: ... }</code>).
Geen HATEOAS	Clients moeten hardcoded endpoints	Overweeg HATEOAS (bijv. <code>_links</code> in responses) voor dynamische navigatie.

3. Security Analyse

✓ Goed Toegepast

- **Authenticatie:** Gebruikt tokens (JWT-achtig) voor endpoints.
- **CORS:** Ingeschakeld met configuratie (`cors.configurationSource`).
- **CSRF:** Uitgeschakeld (`.csrf(AbstractHttpConfigurer::disable)`), wat oké is voor stateless APIs.
- **Password Hashing:** Gebruikt `passwordEncodingService` (waarschijnlijk **BCrypt** of **PBKDF2**).
- **Environment Variables:** Gevoelige data wordt waarschijnlijk via `application.properties` ingesteld.

⚠ Kritieke Verbeterpunten

Risico	Impact	Oplossing
Geen Rate Limiting	Kwetsbaar voor brute-force aanvallen	Implementeer Spring Security Rate Limiting (bijv. <code>Bucket4j</code> of <code>Resilience4j</code>).
Geen Input Validatie	Kwetsbaar voor SQL-injectie/XSS	Gebruik Spring Validation (<code>@Valid</code> , <code>@NotNull</code>) of OWASP Java Encoder .
Geen HTTPS-afdwingen	Man-in-the-middle aanvallen	Configureer HTTPS in <code>application.properties</code> en gebruik een reverse proxy (bijv. Nginx).

Risico	Impact	Oplossing
Zelf-geïmplementeerde JWT	Mogelijke kwetsbaarheden	Gebruik Spring Security OAuth2 of JJWT voor standaard JWT-implementatie.
Geen Secure Headers	Kwetsbaar voor XSS/Clickjacking	Voeg Helmet-achtige headers toe (bijv. Content-Security-Policy , X-Frame-Options).
Geen Logging van Failed Logins	Moeilijk detecteren van aanvallen	Log failed login attempts (bijv. met Logback of SLF4J).
Geen Dependency Scanning	Kwetsbare libraries	Gebruik OWASP Dependency-Check of Snyk om dependencies te scannen.

4. Prioritaire Acties

● Hoog Prioriteit (Security)

1. **Implementeer Rate Limiting** (bijv. **Bucket4j**).
2. **Voeg Input Validatie toe** (Spring Validation + OWASP Encoder).
3. **Forceer HTTPS** in productie.
4. **Vervang zelf-geïmplementeerde JWT** door **JJWT** of Spring Security OAuth2.
5. **Voeg Secure Headers toe** (bijv. **Content-Security-Policy**).

● Middel Prioriteit (REST Best Practices)

1. **Voeg API-versiebeheer toe** (bijv. **/api/v1/**).
2. **Implementeer OpenAPI/Swagger** voor documentatie.
3. **Standaardiseer API-responses** (bijv. **{ data: ..., meta: ... }**).
4. **Voeg Pagination toe** voor endpoints met grote datasets.

● Laag Prioriteit (Optimalisatie)

1. **Overweeg HATEOAS** voor dynamische endpoints.
2. **Log failed login attempts** voor security monitoring.
3. **Scan dependencies** met OWASP Dependency-Check.

5. Voorbeeldcode voor Verbeteringen

1. Rate Limiting (Bucket4j)

```
// pom.xml
<dependency>
  <groupId>com.bucket4j</groupId>
  <artifactId>bucket4j-core</artifactId>
  <version>8.10.0</version>
</dependency>
```

```
// RateLimitConfig.java
@Configuration
public class RateLimitConfig {
    @Bean
    public FilterRegistrationBean<RateLimitFilter> rateLimitFilter() {
        FilterRegistrationBean<RateLimitFilter> registrationBean = new
FilterRegistrationBean<>();
        registrationBean.setFilter(new RateLimitFilter());
        registrationBean.addUrlPatterns("/api/*");
        return registrationBean;
    }
}
```

2. Input Validatie (Spring Validation)

```
// UserController.java
@PostMapping("/register")
public ResponseEntity<UserResponse> register(@Valid @RequestBody UserRequest
userRequest) {
    // ...
}

// UserRequest.java
public class UserRequest {
    @NotBlank(message = "Email is required")
    @Email(message = "Invalid email format")
    private String email;

    @NotBlank(message = "Password is required")
    @Size(min = 8, message = "Password must be at least 8 characters")
    private String password;
}
```

3. Secure Headers (Spring Security)

```
// SecurityConfiguration.java
http
    .headers(headers -> headers
        .contentSecurityPolicy(csp -> csp
            .policyDirectives("default-src 'self'")
        )
        .frameOptions(frame -> frame
            .deny()
        )
    );
```

6. Conclusie

De **ClimateChecker-server** is een **Spring Boot-applicatie** met een **modulaire opzet** en **basale RESTful principes**. Er zijn echter **kritieke security- en REST-best-practice-gaten** die moeten worden aangepakt, met name:

1. **Rate Limiting** en **Input Validatie** (hoogste prioriteit).
2. **HTTPS-afdwingen** en **Secure Headers**.
3. **API-documentatie** (OpenAPI/Swagger) en **versiebeheer**.

Volgende Stappen

- Wil je dat ik **specifieke verbeteringen implementeer** (bijv. rate limiting, input validatie)?
- Of wil je eerst **meer details** over een van de bovenstaande punten?